

this and super

1. Print the fields/instance members of the current class using this and without using object

Answer:

```
class Student {  
    int id = 101;  
    String name = "Naveen";  
    void display() {  
        System.out.println("ID: " + this.id);  
        System.out.println("Name: " + this.name);  
    }  
    public static void main(String[] args) {  
        Student obj = new Student();  
        obj.display();  
    }  
}
```

OUTPUT:

ID: 101

Name: Naveen

2. Print the fields/instance members of the parent class using super

Answer:

```
class Parent {  
    int number = 100;  
}  
class Child extends Parent {  
    int number = 200;  
    void display() {  
        System.out.println("Child Number: " + number);  
        System.out.println("Parent Number: " + super.number);  
    }  
}
```

```

    }

    public static void main(String[] args) {
        Child obj = new Child();
        obj.display();
    }
}

```

Output:

Child Number: 200

Parent Number: 100

3. Call constructor of the current class using this()

Answer:

```

class Student {
    Student() {
        this(101);
        System.out.println("Default Constructor");
    }
    Student(int id) {
        System.out.println("Parameterized Constructor");
    }
    public static void main(String[] args) {
        new Student();
    }
}

```

Output:

Parameterized Constructor

Default Constructor

4. Call argument constructor of current class using this()

Answer:

```
class Student {  
    Student() {  
        this("Naveen");  
    }  
    Student(String name) {  
        System.out.println("Name: " + name);  
    }  
    public static void main(String[] args) {  
        new Student();  
    }  
}
```

Output:

Name: Naveen

5. Call constructor of the parent class using super()

Answer:

```
class Parent {  
    Parent() {  
        System.out.println("Parent Constructor");  
    }  
}  
  
class Child extends Parent {  
    Child() {  
        super();  
    }  
}
```

```
        System.out.println("Child Constructor");
    }

    public static void main(String[] args) {
        new Child();
    }
}
```

Output:

Parent Constructor

Child Constructor

6. Use this() and super() in methods not in constructors

Answer:

Not Possible in Java.

this() and super() are **constructor calls** and **can only be used as the first statement inside a constructor**.

```
void display() {
    this();    // Compile-time error
    super();   // Compile-time error
}
```